

Ejercicio 1: Envenenamiento ARP (ARP Spoofing) y Ataque MITM

Objetivo

El objetivo de esta práctica es llevar a cabo un ataque de envenenamiento ARP (ARP spoofing) utilizando la herramienta Bettercap. El ataque busca manipular las tablas ARP de dos equipos en la red local ("interno1" con IP 192.168.100.11 y "interno2" con IP 192.168.100.22) para que asocien la dirección MAC de un tercer equipo ("observador" con IP 192.168.100.33 y MAC 08:00:27:33:33:33) a las direcciones IP mencionadas. Esto permite que el observador actúe como intermediario (MITM, *Man-in-the-Middle*) interceptando y almacenando el tráfico entre los dos equipos antes de reenviarlo a los destinos legítimos.

Herramientas y Módulos de Bettercap Utilizados

1. **net.probe**: Sonda la red enviando paquetes de prueba para detectar equipos conectados mediante el módulo **net.recon**.

Documentación:

[net.probe](https://www.bettercap.org/modules/ethernet/net.probe/).

2. **net.recon**: Consulta las tablas ARP del sistema para descubrir nuevas máquinas en la red monitorizada.

Documentación:

[net.recon](https://www.bettercap.org/modules/ethernet/net.recon/).

3. **arp.spoof**: Genera paquetes ARP falsificados para realizar el ataque de envenenamiento y posibilitar la interceptación del tráfico.

Documentación:

[arp.spoof](https://www.bettercap.org/modules/ethernet/spoofers/arp.spoof/).

4. **net.sniff**: Actúa como un *sniffer*, capturando el tráfico interceptado por el ataque y retransmitiéndolo a los destinos correspondientes.

Documentación: [net.sniff](https://www.bettercap.org/modules/ethernet/net.sniff/).

Pasos Realizados

1. **Configuración inicial y sondeo de la red**

En el equipo *observador* (192.168.100.33), se inicia Bettercap desde un terminal con el comando:

```
observador:~# bettercap -iface enp0s3
```

- Se activan los módulos `net.recon` y `net.probe` para identificar los dispositivos conectados:

```
192.168.100.0/24 > 192.168.100.33 ☆☆ net.recon on
```

```
192.168.100.0/24 > 192.168.100.33 ☆☆ net.probe on
```

> Tras unos segundos, se detiene el módulo `net.probe`:

```
192.168.100.0/24 > 192.168.100.33 ☆☆ net.probe off
```

```
192.168.100.0/24 > 192.168.100.33 ☆☆ net.show
```

Con este comando, se genera una lista con los dispositivos detectados, mostrando sus IPs, MACs y nombres en la red.

2. ****Captura del tráfico ARP****

En un segundo terminal del equipo *observador*, se utiliza `tshark` para monitorizar el tráfico ARP generado por Bettercap:

```
observador:~# tshark --color -i enp0s3
```

3. ****Ejecución del ataque ARP Spoofing****

Desde el terminal principal de Bettercap, se configuran los parámetros del ataque:

```
192.168.100.0/24 > 192.168.100.33 ☆☆ set arp.spoof.internal true
```

```
192.168.100.0/24 > 192.168.100.33 ☆☆ set arp.spoof.targets
```

```
192.168.100.11,192.168.100.22
```

```
192.168.100.0/24 > 192.168.100.33 ☆☆ arp.spoof on
```

- El parámetro `arp.spoof.internal` asegura que se procesen las respuestas internas en la red local.

- Los objetivos son las IPs 192.168.100.11 e 192.168.100.22.

4. ****Verificación del tráfico interceptado****

Mientras el ataque está en curso, se revisa el tráfico capturado por `tshark` en el segundo terminal del equipo *observador*. Para finalizar la captura, se utiliza `[Ctrl]+C`.

5. ****Consulta de las tablas ARP de los equipos objetivo****

Se inspeccionan las tablas ARP de *interno1* e *interno2* con el comando:

```
interno1:~# arp -n
```

```
interno2:~# arp -n
```

Estas tablas reflejarán la asociación incorrecta entre las direcciones IP e MAC, indicando el éxito del ataque.

Conclusión: Diferencias entre los Tipos de Escaneo (TCP Scan, SYN Scan, NULL Scan)

1. TCP Scan:

- **Descripción:** Este tipo de escaneo establece una conexión completa (*three-way handshake*) con los puertos del objetivo. Si el puerto está abierto, el objetivo responde con un paquete SYN/ACK. Si está cerrado, devuelve un RST.
- **Ventajas:**
 - Fácil de interpretar, ya que sigue el protocolo estándar de comunicación TCP.
 - Compatible con cualquier red que permita conexiones TCP estándar.
- **Desventajas:**
 - Es fácilmente detectable por sistemas IDS/IPS, ya que el establecimiento completo de la conexión deja rastros evidentes en los logs del objetivo.

2. SYN Scan:

- **Descripción:** También conocido como *half-open scan*, este método envía un paquete SYN al puerto objetivo pero no completa el handshake. Si el puerto está abierto, el objetivo responde con un paquete SYN/ACK, pero el atacante responde con un RST para cerrar la conexión.
- **Ventajas:**
 - Más sigiloso que el TCP Scan, ya que no se completa la conexión. Esto reduce los rastros en los logs del sistema objetivo.
 - Más rápido, dado que no necesita completar el handshake.
- **Desventajas:**
 - Aunque es más difícil de detectar, algunos sistemas IDS/IPS están configurados para identificar este tipo de escaneos.

3. NULL Scan:

- **Descripción:** Este tipo de escaneo envía paquetes TCP sin ninguna bandera activada en el encabezado. La respuesta del objetivo depende del sistema operativo:
 - Si el puerto está cerrado, devuelve un RST.
 - Si está abierto, no hay respuesta.
- **Ventajas:**
 - Muy sigiloso, ya que los paquetes no parecen ser parte de una conexión TCP estándar.
 - Útil para evadir ciertos firewalls o IDS que no manejan bien paquetes no estándar.
- **Desventajas:**

- Ineficaz contra sistemas basados en Windows, ya que estos no cumplen con la especificación TCP/IP y responden con un RST independientemente del estado del puerto.
- Más complejo de interpretar, ya que depende del comportamiento específico del sistema operativo del objetivo.

Cuestión 1: Análisis del Ataque

1. **Mensajes ARP enviados por el módulo `arp.spoof`**

Los mensajes ARP generados son falsificados (*spoofed*), indicando que la dirección MAC del equipo *observador* (08:00:27:33:33:33) está asociada a las IPs de *interno1* e *interno2*.

2. **Cambios en las tablas ARP**

Las tablas ARP de *interno1* e *interno2* mostrarán la dirección MAC del equipo *observador* (08:00:27:33:33:33) en lugar de las direcciones MAC reales de sus respectivos destinos. Este cambio implica que todo el tráfico entre estos dos equipos pasará primero por el *observador*, permitiendo la interceptación.

- **Significado de los valores:** Este comportamiento evidencia un ataque exitoso de tipo MITM, donde el atacante logra posicionarse entre los dos objetivos sin que estos perciban la alteración en su comunicación.

Ejercicio 2: Interceptación de mensajes: protocolos en claro vs. protocolos cifrados

Tarea 1: Escucha del protocolo TELNET

Preparación: Se utilizó Bettercap para capturar el tráfico generado durante una conexión TELNET. El tráfico capturado fue almacenado en el archivo `/tmp/telnet.pcap` y posteriormente analizado con Wireshark. Para generar el tráfico, un cliente (interno1) inició sesión en un servidor (interno2) utilizando el comando `telnet`.

Resultados:

1. Los paquetes de la conexión TELNET fueron capturados exitosamente.

2. Al analizar los paquetes en Wireshark, se observó que el protocolo TELNET transmite toda la información en texto claro, incluyendo:
 - Credenciales del usuario (nombre de usuario y contraseña).
 - Comandos ejecutados en el servidor (como `ls -l`).
3. Utilizando la opción “Seguir Flujo TCP” en Wireshark, se visualizó el tráfico bidireccional completo entre cliente y servidor, destacando su falta de cifrado.

Conclusión: TELNET no proporciona ninguna medida de seguridad, lo que lo hace extremadamente vulnerable a ataques de interceptación. Este protocolo no es adecuado para entornos donde la seguridad sea un requerimiento.

Tarea 2: Escucha del protocolo SSH

Preparación: Similar a TELNET, se utilizó Bettercap para capturar tráfico SSH generado por una conexión entre interno1 e interno2. El archivo de captura `/tmp/ssh.pcap` fue analizado con Wireshark.

Resultados:

1. Los paquetes de la conexión SSH fueron capturados correctamente.
2. Al filtrar el tráfico con “ssh” en Wireshark, se identificaron los pasos del intercambio de claves y la negociación del algoritmo:
 - Mensajes de inicio de conexión y configuración del algoritmo.
 - Intercambio Diffie-Hellman para acordar claves secretas.
 - Mensajes “New Keys” que marcan el inicio del tráfico cifrado.
3. Todo el contenido transmitido (incluyendo la contraseña del usuario) estaba cifrado y no pudo ser visualizado en texto claro.

Conclusión: SSH utiliza cifrado robusto para proteger la información transmitida, siendo una alternativa segura frente a TELNET. Este protocolo es ideal para conexiones remotas donde la confidencialidad y la integridad sean esenciales.

Tarea 3: Escucha del protocolo HTTP

Preparación: Se generó tráfico HTTP mediante el acceso a una aplicación web en el servidor interno2, utilizando el navegador Lynx desde interno1. Los datos capturados fueron almacenados en `/tmp/http.pcap` y analizados con Wireshark.

Resultados:

1. Los paquetes de la conexión HTTP fueron capturados exitosamente.

2. Al filtrar por “http” en Wireshark, se visualizó el contenido de las peticiones y respuestas HTTP:
 - Encabezados HTTP con información sobre el navegador y la sesión.
 - Contenido de las solicitudes GET y POST, incluyendo datos sensibles como credenciales.
3. Utilizando “Seguir Flujo HTTP”, se observó claramente todo el tráfico en texto claro, confirmando la falta de cifrado del protocolo HTTP.

Conclusión: HTTP no protege la información transmitida, dejando expuestos datos sensibles como credenciales y sesiones de usuario. Es un protocolo no recomendado para aplicaciones donde se maneje información confidencial.

Tarea 4: Escucha del protocolo HTTPS

Preparación: El tráfico HTTPS se generó accediendo al mismo servidor y aplicación que en la tarea anterior, pero utilizando “https://”. Los paquetes capturados fueron guardados en /tmp/https.pcap.

Resultados:

1. Los paquetes de la conexión HTTPS fueron capturados correctamente.
2. Al filtrar por “tls” en Wireshark, se observó el proceso de negociación de TLS 1.3:
 - Mensajes “Client Hello” y “Server Hello”.
 - Intercambio de claves y establecimiento de conexión segura.
3. Tras la negociación inicial, el tráfico HTTP fue cifrado y no pudo ser visualizado en texto claro, garantizando la confidencialidad de los datos transmitidos.

Conclusión: HTTPS implementa cifrado TLS para proteger el tráfico, ofreciendo una capa de seguridad imprescindible en aplicaciones web. Aunque los datos son capturables, no son legibles sin acceso a las claves privadas del servidor.

Análisis Comparativo y Conclusiones Generales

1. **Vulnerabilidades de Protocolos en Claro:**
 - Tanto TELNET como HTTP exponen información sensible en texto claro, lo que los hace vulnerables a ataques de interceptación.
 - Estos protocolos no deben utilizarse en redes inseguras o públicas.
2. **Ventajas de los Protocolos Cifrados:**
 - SSH y HTTPS utilizan cifrado avanzado (TLS y Diffie-Hellman) para garantizar la confidencialidad e integridad de los datos transmitidos.

- A diferencia de TELNET y HTTP, estos protocolos protegen las credenciales y la información sensible.

3. Recomendaciones:

- Sustituir TELNET por SSH para conexiones remotas seguras.
- Implementar HTTPS en todas las aplicaciones web para proteger las comunicaciones de los usuarios.
- Educar a los administradores sobre las ventajas de los protocolos cifrados frente a los tradicionales en claro.